Integer multiplication below $n \log n$: the exponent $\kappa = 2^{-74}$

October 7, 2026

Abstract

The multitape algorithm of the preprint *Integer multiplication below $n \log n$* [1] multiplies two n-bit integers in time $O(n(\lg n)^{1-\kappa})$ with $\kappa = 2^{-182}$; its constants were chosen conservatively. We show that the same algorithm, with two local changes and new constants, achieves $\kappa = 2^{-74}$. First, the axes of the Gaussian-resampling and Chinese-remainder layouts can be moved with $O(d)$ direct chunk interchanges instead of $O(d^2)$ adjacent ones. This removes the condition $\epsilon(2-\tau) < 1-\tau$ that forced the dimension exponent ϵ below 2^{-50}. Second, the address-interchange network of [1] with ground set of size 46 instead of 100 admits the swap exponent $\tau = 1 - \frac{6}{5}2^{-36}$, against the $1 - 2^{-50}$ used in [1]. These two changes alone give $\kappa = 2^{-76}$. An additive unrolling of the guard-width recursion and a smaller Gaussian width then allow $\epsilon = 3/16$ and give $\kappa = 2^{-74}$. We also show that, with any bit network of the type used in [1], every admissible choice of constants in this architecture has $\kappa < 2^{-73.7}$, so further progress needs a new idea rather than tuning. All conditions and margins are checked in exact rational arithmetic and, independently, in a companion Lean 4 file; the scan over network sizes and the barrier constant are evaluated numerically.

Contents

1 Introduction **2**

2 Moving axes with $O(d)$ interchanges **3**

3 A smaller address-interchange network **4**

4 The guard width **5**

5 A smaller Gaussian width **6**

6 The new constants **6**

7 Limits of parameter tuning **7**

8 Verification **8**

A Exponents of the bit networks **10**

B Verification script **10**

1 Introduction

Throughout, [1] denotes the 73-page preprint dated September 23, 2026, and references such as "Lemma 12", "(38)" or "Table 1" are to its numbering. Our own equations and tables are numbered within sections, as in (2.1) or Table 1.1, so a plain number always refers to [1]. Logarithms log are natural; lg is as in [1].

Theorem 1.1. *Theorem 1 of [1] holds with $\kappa = 2^{-74}$. That is, there is one deterministic Turing machine with a fixed finite alphabet and a fixed finite number of one-dimensional tapes that computes the exact product of two n-bit integers for every $n \geq 1$ in worst-case time $O\big(n(\lg n)^{1-2^{-74}}\big)$. Consequently Corollary 2 (exact division and integer square root) and the transposition bound of Section 11 of [1] also hold with $\kappa = 2^{-74}$.*

The machine is the one constructed in [1], with five differences:

- the schedule of the axis movements in Lemma 20 and Section 9.3 (Section 2);
- the size of the bit network (Section 3);
- the bound on the guard width in Section 5.4 (Section 4);
- the Gaussian width α of Section 9.2.1, with the explicit cutoff $b \geq 2^{24}$ of Sections 9.2.1 and 9.7.1 raised (Section 5);
- the fixed constants, including β, which Section 5.3 fixes to $1/2$ (Section 6).

Every other statement and proof of [1] is used as it stands. Those statements depend on the constants only through the inequalities collected in Section 6. We do not re-verify the parts of [1] that we do not change.

Where 2^{-182} comes from. Section 9.8 of [1] divides the running time by $Tp = \Theta(n)$ and bounds the quotient by seven powers p^{1-g_i}; any $\kappa < \min_i g_i$ is then admissible. With the constants (38), the smallest margin is

$$g_2 = \epsilon c(1-\tau) = 2^{-75} \cdot 2^{-56} \cdot 2^{-50} = 2^{-181},$$

a product of three small numbers.

- $1-\tau$ is the power saving of the chunk interchange (Lemma 12). Section 3.6 of [1] takes $1-\tau = 2^{-50}$. The $h = 100$ network actually gives $1.19 \cdot 2^{-40}$, and the network with $h = 46$ gives $1.24 \cdot 2^{-36}$ (Section 3).
- c is the exponent of the chunk length $K = \lfloor d^c \rfloor$. Proposition 14 requires $\tau(1 + c/\beta) < \lambda < 1$, hence $c < \beta(1-\tau)/\tau$. With $\beta = 1/2$ this forces c below about 2^{-51}, and [1] takes $c = 2^{-56}$.
- ϵ is the exponent of the number of axes $d = \lfloor b^\epsilon \rfloor$. The margin $g_4 = 1 - \tau - \epsilon(2-\tau)$ of the row "CRT and axis layouts" of Table 1 of [1] must be positive, so $\epsilon < (1-\tau)/(2-\tau) < 2^{-50}$. This constraint comes only from the $O(d^2)$ adjacent axis moves in Lemma 20 and Section 9.3.

Table 1.1 records the effect of the changes, applied cumulatively.

Section 7 shows that the coupling between c and τ makes $\kappa \approx \epsilon(1-\tau)^2$ unavoidable in this architecture. Section 8 describes the verification.

Change	$1-\tau$	c	ϵ	$\min_i g_i$	κ
[1], constants (38)	2^{-50}	2^{-56}	2^{-75}	2^{-181}	2^{-182}
$O(d)$ axis interchanges (Sec. 2); new β, c, ϵ	2^{-50}	$\frac{31}{32}2^{-50}$	$\frac{1}{15}$	$2^{-103.95}$	2^{-104}
admissible exponent of the $h = 100$ bit network (Rem. 3.2)	2^{-40}	$\frac{31}{32}2^{-40}$	$\frac{1}{15}$	$2^{-83.95}$	2^{-84}
bit network with $h = 46$ (Sec. 3)	2^{-36}	$\frac{31}{32}2^{-36}$	$\frac{1}{15}$	$2^{-75.95}$	2^{-76}
guard width $C_1 = 5$, smaller α, $\delta = \frac{1}{128}$, and $1-\tau = \frac{6}{5}2^{-36}$ (Secs. 3–5)	$\frac{6}{5}2^{-36}$	$\frac{7}{6}2^{-36}$	$\frac{3}{16}$	$2^{-73.93}$	2^{-74}

Table 1.1: Cumulative effect of the changes. Rows 2–4 use $C_1 = 14$ (which [1] notes is sufficient; it takes $C_1 = 20$), the Gaussian width α of [1] and $\delta = 1/16$. All rows after the first use $\beta = 1 - 2^{-10}$, $\sigma = 1 - 2^{-50}$ and $\lambda' = 1 - 2^{-61}$. Row 2 uses $\lambda = 1 - 2^{-56}$ and the others $\lambda = 1 - 2^{-51}$. In every row the minimum margin is g_2, and κ is the largest power of two below it.

2 Moving axes with $O(d)$ interchanges

Lemma 2.1 (Interchanging two axis fields)**.** *Consider a bit array of volume V whose address consists of fields $F_1, \dots, F_k$, in this physical order, followed by a record suffix. Let $i < j$ and suppose F_i, F_j have widths in $\{\ell - 1, \ell\}$. The address permutation that exchanges the contents of F_i and F_j, so that the old F_j occupies position i and the old F_i position j, and fixes all other fields, can be performed in $O(V(1 + \ell^\tau))$ steps on a fixed number of tapes. The bound includes descriptor work.*

Proof. Collapse the fields before F_i into a prefix $[P]$, the fields strictly between F_i and F_j into a gap $[G]$, and the fields after F_j together with the record suffix into $[B]$. Lemma 12 of [1] accepts arbitrary positive P, G, B. If the two widths are equal, it performs the interchange in $O(V\ell^\tau)$ steps. If they differ by one bit, apply the remark following Lemma 12: write the longer field as a leading bit followed by a field of the shorter width. One equal-width interchange (Lemma 12) and one single-bit move (Lemma 3, $O(V)$) give the required permutation, and the gap stays between the two full fields. The collapsed lengths have $O(\log V)$ bits, and computing them takes $(\log 2V)^{O(1)} = O(V)$ steps. □

Lemma 2.2 (Replacement for the movement bound in Lemma 20)**.** *Under the hypotheses of the second paragraph of Lemma 20 of [1] ($t_i \in \{r/2, r\}$, the full padded box of T entries, $O(p)$ bits per entry), exposing and restoring all the lines of the linewise implementations of $\mathcal{A}$ and $\mathcal{B}_0$ costs*

$$O\big(Tp\,d\,(1 + \ell^\tau)\big).$$

Proof. In the proof of Lemma 20, replace the schedule of adjacent interchanges by the following. To expose axis i, interchange its field with the innermost axis field by Lemma 2.1, unless axis i is already innermost. Then process its lines exactly as in [1] and interchange the two fields back.

The descriptor records which coordinate each physical field holds. Moving fields therefore changes their positions but not the validity conditions, as already noted in the proof of Lemma 20. The line maps write t_i values per line, so the padded box keeps its shape and the field widths never change. After axis i is processed the physical order is the original one. Hence later steps, which expect axis-major order with the length-r axis last, are unaffected.

Each tensor map performs at most $2d$ interchanges, each costing $O(Tp(1 + \ell^\tau))$, and there is a fixed number of tensor maps. The interchanges are exact permutations of the padded box, so they change neither norms nor errors. □

Lemma 2.3 (Replacement in Section 9.3)**.** *The coefficient-address map Φ of Section 9.3 of [1], its inverse and their padding operations cost $O(Tp\,d\,(1+\ell^\tau))$.*

Proof. The only change concerns the step that reorders the complete padded axes to $(a_1,\dots,a_d)$. The nested insertion scan produces the physical order $(a_d,\dots,a_1)$. Reversing it takes the $\lfloor d/2\rfloor$ interchanges of the fields in positions k and $d+1-k$, $1\le k\le\lfloor d/2\rfloor$, each by Lemma 2.1 (widths ℓ or $\ell-1$).

All of these interchanges happen before any triangular update. So when target i is updated, in the order $i=d,\dots,1$, its lower-index controls precede it, as Lemma 4 requires. The inverse first performs the inverse triangular updates ($i=1,\dots,d$) and then the same $\lfloor d/2\rfloor$ interchanges. The d controlled rotations still cost $O(dV)$, and their offset preparation is $o(Tp)$, as in [1]. □

Consequently the row "CRT and axis layouts" of Table 1 of [1] becomes $d(1+\ell^\tau)$. Since $d=\Theta(p^\epsilon)$ and $\ell=\Theta(p^{1-\epsilon})$, its power of p is $\epsilon+\tau(1-\epsilon)$ and its margin is

$$g_4=(1-\tau)(1-\epsilon), \tag{2.1}$$

which is positive for every $0<\epsilon<1$. The condition $\epsilon(2-\tau)<1-\tau$ of Section 9.1 disappears. The binding upper bounds on ϵ come from the guard width ($\epsilon C_1<1$) and the Gaussian row (Sections 4 and 5).

3 A smaller address-interchange network

The bit network of Section 3.1 of [1] is defined for every ground-set size h. [1] uses $h=100$ for both networks, but notes in Section 5.3 that the constants of the complex network "need not equal the constants used to prove the chunk-swap lemma". We keep the complex (phase) network at $h=100$, with $\sigma=1-2^{-50}$ as in (10), and use the bit network with $h=46$.

For general h the bit construction has

$$v=\tbinom{h}{3},\quad N=v^3,\quad m=h^3,\quad I=3v^2,\quad z_{\mathrm b}=3\tbinom{h-3}{2},\quad W_{\mathrm b}=2N+I(vz_{\mathrm b}+h),\quad L_{\mathrm b}=Ih^2,$$

and $s_{\mathrm b}=W_{\mathrm b}m-N+2L_{\mathrm b}$ as in Proposition 7. Nothing in Sections 3.1–3.4 of [1] depends on $h=100$ except the printed numbers. The form $x^{\mathsf T}(I-J/9)y$ is nondegenerate whenever $h\ne 9$ (its eigenvalues are 1 and $1-h/9$). Every label in table (6) and in the proof of Lemma 6 is built from lines spanned by vectors of nonzero norm (the triple indicators satisfy $\langle t_T,t_T\rangle=2$, and neighboring indicators are orthogonal), using orthogonal complements, tensor products and orthogonal sums. So all labels are nondegenerate, and Lemma 6 and Proposition 7 hold with these counts. The binary orthonormal-basis assertion of Lemma 6 concerns only the complex network. The rank saving is $W_{\mathrm b}m-s_{\mathrm b}=N-2L_{\mathrm b}=v^2(v-6h^2)$, which is positive if and only if $h\ge 39$.

For $h=46$:

$$\begin{aligned}
&v=15180,\qquad N=3\,497\,963\,832\,000,\qquad m=97\,336,\qquad I=691\,297\,200,\qquad z_{\mathrm b}=2709,\\
&W_{\mathrm b}=28\,434\,979\,789\,999\,200,\qquad L_{\mathrm b}=1\,462\,784\,875\,200=\tfrac{23}{55}N,\\
&s_{\mathrm b}=2\,767\,747\,192\,266\,968\,049\,600,\qquad W_{\mathrm b}m=2\,767\,747\,192\,839\,362\,131\,200,\\
&\eta_{\mathrm b}:=\frac{W_{\mathrm b}m-s_{\mathrm b}}{W_{\mathrm b}m}=\frac{9}{43\,518\,487\,588}=2.0681\ldots\cdot 10^{-10}.
\end{aligned}$$

Lemma 3.1. *For the bit network with $h=46$ and $\tau=1-\frac{6}{5}2^{-36}$, and* a fortiori *for $\tau=1-2^{-36}$, we have $s_{\mathrm b}/W_{\mathrm b}<m^\tau$. So Proposition 11 and Lemma 12 hold with this τ.*

Proof. Put $T = 1 - \tau$. Since $m < 2^{17}$ and $\log 2 < 0.6932$, we have $\log m < 11.7844$, and therefore

$$T \log m < \tfrac{6}{5} \cdot 11.7844 \cdot 2^{-36} < 2.058 \cdot 10^{-10} < \eta_{\mathrm{b}}.$$

Hence $m^{\tau} = m \exp(-T \log m) > m(1 - T \log m) > m(1 - \eta_{\mathrm{b}}) = s_{\mathrm{b}}/W_{\mathrm{b}}$. □

Remark 3.2. We record three facts about the two networks.

1. The exact exponent is $\log_m(s_{\mathrm{b}}/W_{\mathrm{b}}) = 1 - T^*(46)$ with $T^*(46) = 1.23732\ldots \cdot 2^{-36}$. Over all ground-set sizes, $T^*(h) := -\log(1 - \eta_{\mathrm{b}}(h))/\log h^3$ is largest at $h = 46$; see Table A.1 in Appendix A.

2. Even the $h = 100$ bit network of [1] has $T^*(100) = 1.194\ldots \cdot 2^{-40}$, so $\tau = 1 - 2^{-40}$ is admissible there. Section 3.6 of [1] uses only $1 - 2^{-50}$.

3. Lemma 10, Proposition 11 and Lemma 12 are generic in (m, W, s). Only the finite rational table of Lemma 10 and its prime q must be recomputed for the matrices of the new network. In Section 5 of [1], τ enters only through the cost $O(V(L^{\tau} + 1))$ of Lemma 13, that is, through the conditions $\max\{\tau, \sigma\} < \lambda$ and $\tau(1 + c/\beta) < \lambda$ of Proposition 14.

4 The guard width

Section 5.4 of [1] bounds the arithmetic depth of one simultaneous layer by the recursion

$$A(e) \le s\, A(e/m) + E \quad \text{(internal node)}, \qquad A(e) \le 8e \quad \text{(leaf, } e < d^{\beta}\text{)}, \tag{4.1}$$

where m, $s = s_{\mathrm{c}}$ and $W = W_{\mathrm{c}}$ are the constants of the complex network ($h = 100$) and $E = 64(W + m + 1)^3$. It then uses $sx + E \le (s + E)x = Bx$, which replaces the per-level factor $s \approx m^{4.55}$ by $B \approx m^{10.94}$ and leads to the exponent $\nu + 3 = 14$ with $\nu = 11$ ([1] takes $C_1 = 20$). Unrolling the recursion additively loses much less.

Lemma 4.1. *Let $M \ge 1$ and $C \ge 1$ be real numbers, let $E \ge 0$, and let $2 \le \mu \le M^C$. Suppose $A_0, A_1, \ldots$ satisfy $A_j \le 8M^j$ at leaf levels (with 0 a leaf level) and $A_j \le \mu A_{j-1} + E$ at all other levels. Then $A_j + E \le (8 + E)M^{Cj}$ for all j.*

Proof. Induction on j. At a leaf level, $A_j + E \le 8M^j + E \le (8 + E)M^{Cj}$. At any other level, $A_j + E \le \mu A_{j-1} + 2E \le \mu(A_{j-1} + E) \le M^C(8 + E)M^{C(j-1)}$, using $\mu \ge 2$. □

Corollary 4.2 (Guard exponent $C_1 = 5$)**.** *Section 5.4 and Proposition 14 of [1] hold with $C_1 = 5$, that is, with $\Delta = \lceil C_0 d^5 \rceil$ and the same $C_0 = 128mB^2$. The only resulting condition is $\epsilon C_1 < 1$.*

Proof. For the $h = 100$ complex network, $2 \le s_{\mathrm{c}} = 1\,873\,807\,244\,636\,671\,267\,308\,000\,000 < 10^{30} = m^5$. By Lemma 4.1, applied to (4.1) with $A_j = A(m^j)$, a piece of m^j chunks has depth at most $(8+E)m^{5j}$. Write $D - q_0 = \sum_j a_j m^j$ as in [1]. Since $\sum x_i^5 \le (\sum x_i)^5$, the pieces together contribute at most $(8 + E)(D - q_0)^5 \le (8 + E)d^5$. At most d individually processed kernels add $8d$, and the outer diagonals and the scalar b_D add $t + 9 \le d + 9$. The depth of a complete layer is therefore at most $(26 + E)d^5 \le C_0 d^5$, because $C_0 = 128mB^2 \ge 26 + E$. The remainder of Section 5.4 uses C_1 only through $\Delta \le p$, that is, $\epsilon C_1 < 1$. □

Remark 4.3 (Guard exponent $1 + 2^{-10}$). The recursion (4.1) concatenates all s child calls of a network invocation, but a dependency chain meets far fewer.

A chain meets only the child calls on the edges of the wire it currently occupies, and it changes wires only at gates, where all incident wires carry the same label. So the number of child calls a chain meets at one level is the total variation of the label dimension along a path in the label graph given by table (6) and Lemma 6.

Let $k(x)$ count the decreasing center edges ($3 \to 4$, one per stage) that can precede the vertex x, and put $\Phi(x) = \dim x + 2h\,k(x)$. Along every edge, Φ increases by at least the absolute change in dimension, and $0 \le \Phi \le m + 6h$. Hence every chain meets at most $m + 6h$ child calls per level, and this bound is attained.

Lemma 4.1 with $\mu = m + 6h \le m^{1+2^{-10}}$ then gives the guard exponent $1 + 2^{-10}$, and with $\mu \le m^2$ it gives $C_1 = 2$. We do not need this sharper bound.

5 A smaller Gaussian width

Lemma 19 of [1], which adapts the Gaussian resampling of Harvey and van der Hoeven [2, Theorem 4.2, Proposition 4.7 and Lemmas 4.8–4.12], requires $2 \le \alpha < \sqrt{p}$ and $\theta > p/\alpha^4$, and in Section 9.2.2 every $\theta_i = t_i/s_i - 1$ exceeds $\eta = 1/(4d)$. Hence

$$\alpha := \lceil (24db)^{1/4} \rceil \tag{5.1}$$

suffices, since $\alpha^4\theta_i > 24db/(4d) = 6b = p$; and $2 \le \alpha < \sqrt{p}$ for all large n. [1] takes $\alpha = \lceil (12d^2b)^{1/4} \rceil$, which carries an extra factor $d/2$ inside the fourth root. With (5.1):

- $\alpha = \Theta(p^{(1+\epsilon)/4})$, so the row "Gaussian line maps" $dp^{1/2+\delta}\alpha$ of Table 1 of [1] has power $3/4 + \delta + 5\epsilon/4$ and margin $g_5 = 1/4 - \delta - 5\epsilon/4$;
- $\gamma = 2d\alpha^2 \le 8\sqrt{24}\, d^{3/2}b^{1/2} < 40\, b^{1/2+3\epsilon/2}$. This is $o(b)$ for $\epsilon < 1/3$, so (44) holds for all large n. The explicit cutoff $b \ge 2^{24}$ in Sections 9.2.1 and 9.7.1 must be raised; for $\epsilon = 3/16$, $b \ge 2^{34}$ gives $\gamma \le b/4$;
- the Neumann count $k \le \alpha^2 + 1 < p + 1$ and the cost bound $O(tp^{3/2+\delta}\alpha)$ of Lemma 19 are unchanged, since their proofs use only $\alpha < \sqrt{p}$ and $\alpha^4\theta > p$.

6 The new constants

Replace (38) of [1] by

$$\begin{gathered} \tau = 1 - \tfrac{6}{5}2^{-36} \ \text{(bit network, } h = 46), \qquad \sigma = 1 - 2^{-50} \ \text{(complex network, } h = 100), \\ \beta = 1 - 2^{-10}, \qquad c = \tfrac{7}{6}2^{-36}, \qquad \lambda = 1 - 2^{-51}, \qquad \lambda' = 1 - 2^{-61}, \\ C_1 = 5, \qquad \epsilon = \tfrac{3}{16}, \qquad \delta = \tfrac{1}{128}, \qquad \kappa = 2^{-74}. \end{gathered} \tag{6.1}$$

Section 5.3 and the statement of Proposition 14 of [1] fix $\tau = \sigma = 1 - 2^{-50}$, $\beta = 1/2$ and $C_1 = 20$. The proofs are parametric in these constants (for C_1, use Corollary 4.2), and we read Proposition 14 with the values (6.1). Its conditions hold:

$$\max\{\tau, \sigma\} = 1 - 2^{-50} < \lambda < 1, \qquad \tau(1 + c/\beta) < 1 - 2^{-41} < \lambda,$$

$$\sigma + \beta(1 - \sigma) = 1 - 2^{-60} < \lambda' < 1, \qquad \lambda < \lambda', \qquad \epsilon C_1 = \tfrac{15}{16} < 1.$$

The remaining conditions of Section 9 also hold:

- $0 < \delta < 1/8$ (Lemma 19);
- $\epsilon < 1/3$ ($\gamma = o(b)$, Section 5);
- $\epsilon < 1/2$ (prime search: $\log(r/2)/d = \Theta(p^{1-2\epsilon}) \to \infty$);
- $\epsilon c > 0$ ($K/\log p \to \infty$);
- $K = \lfloor d^c \rfloor \le \ell - 1$ for large n.

Operation	Normalized cost	Power of p
Prefix-slot moves and individual butterfly rounds	dK	$\epsilon(1+c)$
Chunk exchanges for transform layout	$pK^{\tau-1}$	$1-\epsilon c(1-\tau)$
Simultaneous butterfly rounds	$\ell d^{\lambda'}$	$1-\epsilon(1-\lambda')$
CRT and axis layouts (Sec. 2)	$d(1+\ell^{\tau})$	$1-(1-\tau)(1-\epsilon)$
Gaussian line maps (Sec. 5)	$dp^{1/2+\delta}\alpha$	$3/4+\delta+5\epsilon/4$
Chirps, twists and scalar products	dp^{δ}	$\epsilon+\delta$
Packed polynomial products	$\log(rp)$	$1-\epsilon$
Final scaling and rounding	p^{δ}	absorbed above
Input, padding, carries and output	1	absorbed above

Table 6.1: Replacement for Table 1 of [1]: costs divided by Tp.

The margins of the seven dominant rows of Table 6.1 are

$$\begin{aligned} g_1 &= 1-\epsilon(1+c) = \tfrac{13}{16} - \tfrac{7}{32}2^{-36}, & g_2 &= \epsilon c(1-\tau) = \tfrac{21}{80}\,2^{-72} = 1.05\cdot 2^{-74}, \\ g_3 &= \epsilon(1-\lambda') = \tfrac{3}{16}2^{-61}, & g_4 &= (1-\tau)(1-\epsilon) = \tfrac{39}{40}2^{-36}, \\ g_5 &= \tfrac{1}{4} - \delta - \tfrac{5}{4}\epsilon = \tfrac{1}{128}, & g_6 &= 1-\delta-\epsilon = \tfrac{103}{128}, \qquad g_7 = \epsilon = \tfrac{3}{16}. \end{aligned} \tag{6.2}$$

Thus $\min_i g_i = g_2 > 2^{-74} = \kappa$.

Proof of Theorem 1.1. Follow Section 9.8 of [1] with Table 6.1 in place of Table 1 of [1]. Each row is $O(p^{1-g_i})$: the factor $\log(2d)$ from the base-m groups is absorbed in λ', and $\log p$ in p^{δ}. Since $\kappa < \min_i g_i$, any fixed power of $\log p$ is at most $p^{\min_i g_i - \kappa}$ for large p. The factor two in "$\min_i g_i = 2\kappa$" of [1] is not needed, although here it would cost only one bit.

Hence the large-input cost is $O(Tp \cdot p^{1-\kappa}) = O(n(\lg n)^{1-\kappa})$. The finitely many small lengths are handled by schoolbook multiplication, as in [1]. Corollary 2 and Section 11 use only the multiplication bound and $0 < \kappa < 1$. □

Remark 6.1 ($\kappa = 2^{-76}$ from Sections 2 and 3 alone). Use the guard exponent $C_1 = 14$, which [1] notes is sufficient (it takes $C_1 = 20$); keep its Gaussian width α and $\delta = 1/16$. Take $\tau = 1 - 2^{-36}$ ($h = 46$), $c = \frac{31}{32}2^{-36}$ and $\epsilon = 1/15$, with $\sigma, \beta, \lambda, \lambda'$ as in (6.1). All conditions hold, including $\epsilon < 1/12$ as used in (40). The margins have minimum $g_2 = \frac{31}{480}2^{-72} > 2^{-76}$, with $g_5 = 1/4 - \delta - 3\epsilon/2 = 7/80$, so $\kappa = 2^{-76}$.

7 Limits of parameter tuning

Proposition 7.1. *Keep the algorithm of [1] with the changes above and its cost accounting (Table 6.1). Then every admissible choice of constants satisfies $\kappa < \epsilon(1-\tau)^2/\tau$ and $\epsilon < 1/5$. If the*

interchange network is the bit network of Section 3.1 of [1] for some ground-set size h, then also $1-\tau < 1.2374 \cdot 2^{-36}$, and consequently $\kappa < 2^{-73.70}$.

Proof. The condition $\tau(1+c/\beta) < \lambda < 1$ of Proposition 14 gives $c < \beta(1/\tau - 1) < (1-\tau)/\tau$, where $\beta < 1$ because $\sigma + \beta(1-\sigma) < \lambda' < 1$; so $\kappa < g_2 = \epsilon c(1-\tau) < \epsilon(1-\tau)^2/\tau$. The condition $g_5 > 0$ gives $\epsilon < 1/5 - 4\delta/5 < 1/5$; with the Gaussian width of [1] it gives even $\epsilon < 1/6$.

Proposition 11 needs $s/W < m^\tau$, that is, $1-\tau < T^*(h)$. For $h < 39$ there is no rank saving. For $39 \le h \le 2000$ a numerical computation (Appendix B; selected values in Table A.1) gives $T^*(h) \le T^*(46) < 1.2374 \cdot 2^{-36}$. For $h > 2000$,

$$\eta_{\mathrm{b}} \le \frac{N}{W_{\mathrm{b}} m} \le \frac{N}{I v z_{\mathrm{b}} m} = \frac{1}{3 z_{\mathrm{b}} m},$$

and therefore $T^*(h) < 2\eta_{\mathrm{b}}/\log m < 10^{-18}$. Finally $\frac{1}{5}(1.2374 \cdot 2^{-36})^2/(1 - 1.2374 \cdot 2^{-36}) < 2^{-73.70}$. □

The obstruction is structural. The chunk length $K = \lfloor d^c \rfloor$ serves two purposes. In the transform layout of Lemma 16 there are $O(p/K)$ chunk interchanges of width K, with normalized cost $pK^{\tau-1}$, and this term wants K large. In each simultaneous layer, the packed selected-bit additions of Lemma 13 interchange slots of width $(e/m)K$, at cost $O((eK)^\tau)$ per node, and this forces $c < (1-\tau)/\tau$. Going substantially below 2^{-74} therefore needs a new idea. Two possibilities are an address-interchange network with a larger rank saving (the saving enters squared), or a layout whose chunk length is not tied to the swap exponent, for instance a way to realize the layout permutation of Lemma 16 at cost below $pK^{\tau-1}$.

8 Verification

Exact arithmetic. The script in Appendix B (Python) checks every condition with integers and fractions; floating point is used only for displayed logarithms and for the scan over h. It does the following:

- recomputes the printed constants of [1] for $h = 100$;
- computes the $h = 46$ counts and η_{b};
- checks the exponent certificates of Lemma 3.1 and of Remark 3.2, and the optimality of $h = 46$ for $h \le 2000$;
- checks every condition and margin for the five rows of Table 1.1, including $\epsilon < 1/12$ for the four rows that keep the Gaussian width of [1];
- checks $8\sqrt{24} < 40$ and that $40\, b^{25/32} \le b/4$ for $b \ge 2^{34}$ (Section 5);
- evaluates the barrier of Proposition 7.1.

Lean. The companion file `IntMulKappa.lean` checks the same arithmetic independently in Lean 4 [3]. It uses core Lean only (no Mathlib), version 4.34.1, the toolchain pinned by the `openai/math` repository. It contains no `sorry` and no `native_decide`, and `#print axioms` reports at most `propext`, `Classical.choice` and `Quot.sound`. It compiles in a few seconds with `lean IntMulKappa.lean`. Its main statements are listed in Table 8.1.

Lean theorem	Content
`printed_values_h100`, `counts_h46`, `relations_h46`, `eta_b_h46`	the constants of Section 3 and of [1] (3)–(4), (9), Proposition 8
`rate_cert`	if $Q, Y > 0$ and $Q^q M^p < y_n^q$, where $y_0 = \lfloor QX/Y \rfloor$ and $y_{i+1} = \lfloor y_i^2/Q \rfloor$, then $Y^{q2^n} M^p < X^{q2^n}$
`tau_B`, `tau_A`	$s_{\mathrm{b}}^K m^6 < (W_{\mathrm{b}} m)^K$ with $K = 5 \cdot 2^{36}$, and $s_{\mathrm{b}}^{2^{36}} m < (W_{\mathrm{b}} m)^{2^{36}}$ $(h = 46)$; these are equivalent to $s_{\mathrm{b}}/W_{\mathrm{b}} < m^{1-\frac{6}{5}2^{-36}}$ and $s_{\mathrm{b}}/W_{\mathrm{b}} < m^{1-2^{-36}}$
`tau_100`, `sigma_paper`	the same for the $h = 100$ bit network with $1 - 2^{-40}$, and for the $h = 100$ complex network with $\sigma = 1 - 2^{-50}$
`depth_unroll`, `sum_pow_le`	Lemma 4.1 for integer C, and $\sum x_i^C \le (\sum x_i)^C$
`guard_instances_h100`	$2 \le s_{\mathrm{c}} \le m^5$ and $m + 6h \le m^2$ for $h = 100$
`chain_tv_bound` $(h = 100)$, `chain_tv_bound_25`, `chain_tv_bound_46`, `extremal_tv`	in the label graph of Remark 4.3, every path has total variation at most $h^3 + 6h$, and one path attains it
`SetB.certified`, `SetA.certified`, `PaperSet.certified`	all conditions and margins of (6.1), of Remark 6.1, and of (38) of [1] $(\min g = g_2 = 2^{-181})$

Table 8.1: Main statements of `IntMulKappa.lean`.

The exponents $q2^n$ in `tau_B` and the other certificates are far too large to evaluate. These statements are proved by applying `rate_cert`, and the kernel only checks the small final inequality, which needs 36 to 50 truncated squarings at scale $Q = 2^{160}$.

The Lean file certifies the exact arithmetic of the improvement: the network counts, the exponent certificates, and all conditions and margins. The scan over h and the barrier constant of Proposition 7.1 are numerical (Appendix B). The file does not formalize the machine model, the algorithm of [1], or Lemmas 2.1–2.3, and the correspondence between dependency chains and paths in the label graph is part of the informal argument.

At the time of writing, the `openai/math` repository (commit `adc7f12`) contains no formalization of [1] (family 109). Its formalized Fourier result, `OAI.ExactFourier.main_theorem` (family 130), is a different statement in a different model: a subsequential gate-count bound for exact complex Fourier circuits (for every $c > 0$ there are arbitrarily large n with a DFT circuit of fewer than $cn \log_2 n$ gates).

A Exponents of the bit networks

h	$\eta_{\mathrm{b}}(h)$	$T^*(h)/2^{-36}$	h	$\eta_{\mathrm{b}}(h)$	$T^*(h)/2^{-36}$
39	$4.228 \cdot 10^{-12}$	0.0264	47	$2.065 \cdot 10^{-10}$	1.2288
40	$7.385 \cdot 10^{-11}$	0.4586	48	$2.037 \cdot 10^{-10}$	1.2052
41	$1.234 \cdot 10^{-10}$	0.7614	49	$1.990 \cdot 10^{-10}$	1.1710
42	$1.579 \cdot 10^{-10}$	0.9678	50	$1.929 \cdot 10^{-10}$	1.1298
43	$1.810 \cdot 10^{-10}$	1.1023	51	$1.861 \cdot 10^{-10}$	1.0840
44	$1.955 \cdot 10^{-10}$	1.1833	52	$1.786 \cdot 10^{-10}$	1.0356
45	$2.035 \cdot 10^{-10}$	1.2248	100	$1.501 \cdot 10^{-11}$	0.0747
46	$2.068 \cdot 10^{-10}$	**1.2373**			

Table A.1: Rank-saving ratio $\eta_{\mathrm{b}}(h) = (N - 2L_{\mathrm{b}})/(W_{\mathrm{b}}m)$ and exact swap exponent $1 - T^*(h)$, $T^*(h) = -\log(1-\eta_{\mathrm{b}})/\log h^3$, of the bit network of [1], Section 3.1, near the optimum. For $h \le 38$ there is no saving. The value at $h = 100$ is $T^*(100) = 1.194 \cdot 2^{-40}$.

B Verification script

```
"""Verification of the constants in the note "Integer multiplication below n log n:
the exponent kappa = 2^-74".  All conditions and margins use exact integer/Fraction
arithmetic; floating-point logarithms are used only for display and for the scan of
T*(h) over h.  Run: python3 verify_kappa.py"""
from fractions import Fraction as F
from math import comb, log, log2

def bit_net(h):          # Section 3.1 of [OAI], bit construction, ground set [h]
    v = comb(h, 3); N = v**3; m = h**3; I = 3*v*v
    W = 2*N + I*(v*3*comb(h-3, 2) + h); L = I*h*h
    return dict(v=v, N=N, m=m, W=W, L=L, s=W*m - N + 2*L)
def cx_net(h):           # Section 3.1 of [OAI], complex construction
    v = comb(h, 3); N = v**3; m = h**3; I = 3*v*v
    W = 2*N + I*(v*(comb(h-3, 3) + 3*(h-3)) + h + 1); L = I*(h+1)*h
    return dict(v=v, N=N, m=m, W=W, L=L, s=W*m - 2*N + 2*L)
def eta(d): return F(d['W']*d['m'] - d['s'], d['W']*d['m'])
def Tstar(d):            # 1 - log_m(s/W) = -ln(1-eta)/ln m  (floating point, for display)
    e = eta(d); return -log(1 - e.numerator/e.denominator)/log(d['m']) if e > 0 else float('nan')

# --- the paper's printed values (h = 100) ------------------------------------------------
b100, c100 = bit_net(100), cx_net(100)
assert (b100['W'], b100['s']) == (177176569091445000000, 17717656908878586128700000)
assert (c100['W'], c100['s']) == (1873807244643542670000, 18738072446366712673080000000)
assert eta(b100) == F(339, 22587335000000) and eta(c100) == F(73, 19906842167500)

# --- the h = 46 bit network --------------------------------------------------------------
b = bit_net(46)
assert (b['v'], b['N'], b['m']) == (15180, 3497963832000, 97336)
assert (b['W'], b['L'], b['s']) == (28434979789999200, 1462784875200, 2767747192266968049600)
assert F(b['L'], b['N']) == F(23, 55) and eta(b) == F(9, 43518487588)
print("h=46: W_b m =", b['W']*b['m'], "  eta_b =", eta(b), " T* = %.5f * 2^-36" % (Tstar(b)*2**36))

# exponent certificates:  T ln m < eta  ==>  m^(1-T) > m(1 - T ln m) > m(1-eta) = s/W
ln2_up = F(6932, 10000); assert log(2) < 0.6932
def cert(T, d, ln_m_up): return T*ln_m_up < eta(d)
assert b['m'] < 2**17 and cert(F(6, 5)/2**36, b, 17*ln2_up)          # tau = 1-(6/5)2^-36
assert cert(F(1, 2**36), b, 17*ln2_up)                               # tau = 1-2^-36
assert b100['m'] < 2**20 and cert(F(1, 2**40), b100, 20*ln2_up)      # h=100: tau = 1-2^-40
assert cert(F(1, 2**50), c100, 20*ln2_up)                            # sigma = 1-2^-50 (paper)

# optimality of h = 46 among the bit networks (deficit N - 2L > 0 needs h >= 39)
```

```
assert all(bit_net(h)['N'] <= 2*bit_net(h)['L'] for h in range(7, 39))
best = max(range(39, 2001), key=lambda h: Tstar(bit_net(h)))
assert best == 46
print("T*(h) for h = 39..52, 100:", {h: round(Tstar(bit_net(h))*2**36, 4) for h in list(range(39, 53)) +
    [100]})

# guard width: s_c <= m^5 (C1 = 5) and the chain-variation route m + 6h <= m^(1+2^-10)
assert 2 <= c100['s'] <= c100['m']**5 and (1 + 600/10**6)**1024 <= 10**6

# --- parameter sets ------------------------------------------------------------------
def check(name, T, S, beta, c, lam, lamp, C1, eps, delta, a5, kappa, axis_linear=True):
    tau, sig = 1 - T, 1 - S
    eps_max = F(1, 12) if a5 == F(3, 2) else F(1, 3)   # (40) of [OAI] / gamma = o(b) with the new alpha
    conds = [max(tau, sig) < lam < 1, tau*(1 + c/beta) < lam, max(lam, sig + beta*(1 - sig)) < lamp < 1,
             eps*C1 < 1, 0 < delta < F(1, 8), 0 < c, 0 < beta < 1, eps < eps_max]
    g = [1 - eps*(1 + c), eps*c*T, eps*(1 - lamp),
         T*(1 - eps) if axis_linear else 1 - tau - eps*(2 - tau),
         F(1, 4) - delta - a5*eps, 1 - delta - eps, eps]
    assert all(conds), name
    assert all(x > 0 for x in g) and kappa < min(g), name
    print(f"{name:7s} min g = 2^{log2(float(min(g))):.4f}  (g2 = {g[1]*2**72} * 2^-72),  kappa = 2^{log2(
     float(kappa)):.0f}")
    return min(g)
check("paper", F(1, 2**50), F(1, 2**50), F(1, 2), F(1, 2**56), 1 - F(1, 2**52), 1 - F(1, 2**54),
      20, F(1, 2**75), F(1, 16), F(3, 2), F(1, 2**182), axis_linear=False)
beta, lam, lamp, S = 1 - F(1, 2**10), 1 - F(1, 2**51), 1 - F(1, 2**61), F(1, 2**50)
check("step1", F(1, 2**50), S, beta, F(31, 32)/2**50, 1 - F(1, 2**56), lamp, 14, F(1, 15), F(1, 16), F(3, 2),
      F(1, 2**104))
check("step2", F(1, 2**40), S, beta, F(31, 32)/2**40, lam, lamp, 14, F(1, 15), F(1, 16), F(3, 2), F(1, 2**84)
    )
check("Set A", F(1, 2**36), S, beta, F(31, 32)/2**36, lam, lamp, 14, F(1, 15), F(1, 16), F(3, 2), F(1, 2**76)
    )
gB = check("Set B", F(6, 5)/2**36, S, beta, F(7, 6)/2**36, lam, lamp, 5, F(3, 16), F(1, 128), F(5, 4), F(1,
    2**74))
assert gB == F(21, 80)/2**72
TB, cB = F(6, 5)/2**36, F(7, 6)/2**36
assert (1 - TB)*(1 + cB/beta) < 1 - F(1, 2**41)

# --- smaller Gaussian width: gamma = 2 d alpha^2 <= 8 sqrt(24) d^(3/2) b^(1/2) < 40 b^(25/32)
assert 64*24 < 40**2                        # 8 sqrt(24) < 40
assert 160**32 <= (2**34)**7                # b >= 2^34  ==>  40 b^(25/32) <= b/4  (i.e. b^(7/32) >= 160)

# --- the barrier: g2 < eps * T^2/(1-T) with eps < 1/5 and T < T*(46) ---------------------
Ts = Tstar(b)
print("barrier: (1/5) T*^2/(1-T*) = 2^%.3f" % log2(Ts*Ts/5/(1 - Ts)))
print("all checks passed")
```

References

[1] OpenAI. *Integer multiplication below $n \log n$*. OpenAI Math Release preprint OAI:Integer-multiplication-below-n-log-n-September-23-2026, September 23, 2026. https://github.com/openai/math.

[2] D. Harvey and J. van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics* 193(2):563–617, 2021. Result numbering as in the 45-page author-hosted manuscript, following [1].

[3] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pp. 625–635, Springer, 2021.